# String method

## String Methods

let msg = "  Hello  ";

str.trim()

Trims whitespaces from both ends of string & returns a new one.

```
> let msg = "   Hello   ";
<· undefined
> msg.trim();
<· 'Hello'
> msg
<· '   Hello   '
```



output : "Hello", but value of msg remains same.

## String are Immutable in JS

No changes can be made to strings.

Whenever we do try to make a change, a new string is created and old one remains same.

# String Methods

```
let str = "Random string";
```

```
str.toUpperCase( )        "RANDOM STRING"

str.toLowerCase( )        "random string"
```

# String Methods with Arguments

Argument is a some value that we pass to the method.

**Format**

```
stringName.method( arg )
```

## indexOf

Returns the first index of occurrence of some value in string. Or gives -1 if not found.

```
let str = "IloveCoding";
          /
          0
```



```
str.indexOf("love")      // 1

str.indexOf("J")         // -1 (not found)

str.indexOf("o")         // 2 (only 1 index)
```

## Method Chaining

Using one method after another. Order of execution will be left to right.

```
str.toUpperCase( ).trim( )
```

## slice

Returns a part of the original string as a new string.

```
let str = "IloveCoding";

    str.slice(5)              // "Coding"

    str.slice(1, 4)           // "love"

    str.slice(-num)  =  str.slice(length-num)
```

## replace

Searches a value in the string & returns a new string with the value replaced.

```
let str = "IloveCoding";

    str.slice("love", "do")   // "IdoCoding"

    str.slice("o", "x")       // "IlxveCoding"
```

**Str.replace(“love”,”do”); // o/p= ‘idocoding’**

# repeat

Returns a string with the number of copies of a string

```
let str = "Mango";

    str.repeat(3)              // "MangoMangoMango"
```

# Creating Arrays

```
let marks = [99, 85, 93, 76, 62];
let names = ["adam", "bob", "catlyn"];
let info = ["aman", 25, 6.1];   //mixed array

//empty array
let newArr = [];
```

arr.length

# Arrays are Mutable

```
> let fruits = ["mango", "apple", "litchi"];
< undefined
> fruits[0] = "banana";
< 'banana'
> fruits
< ▶ (3) ['banana', 'apple', 'litchi']
```

# Array Methods

**Push** : add to end

**Unshift** : add to start

**Pop** : delete from end & returns it

**Shift** : delete from start & returns it

# Array Methods

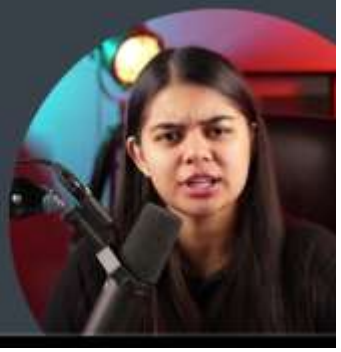
**indexOf** : returns index of something

```
> let primary = ["red", "yellow", "blue"];
```

```
> primary.indexOf("yellow");
< 1
> primary.indexOf("green");
< -1
> primary.indexOf("Yellow");
< -1
```

**includes** : search for a value

```
> primary.includes("red");
< true
> primary.includes("green");
< false
```





# Array Methods

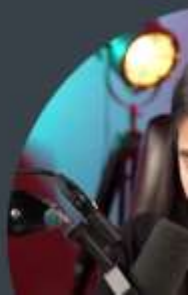
```
> let primary = ["red", "yellow", "blue"];
< undefined
> let secondary = ["orange", "green", "violet"];
< undefined
```

**concat** : merge 2 arrays

```
> primary.concat(secondary);
< ▶ (6) ['red', 'yellow', 'blue', 'orange', 'green', 'violet']
```

**reverse** : reverse an array

```
> primary.reverse();
< ▶ (3) ['blue', 'yellow', 'red']
```



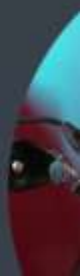
slice ( start

# Array Methods

**slice : copies a portion of an array**

```
> let colors = ["red", "yellow", "blue", "orange", "pink", "white"];
```

```
> colors.slice()
< ▶ (6) ['red', 'yellow', 'blue', 'orange', 'pink', 'white']
> colors.slice(2);
< ▶ (4) ['blue', 'orange', 'pink', 'white']
> colors.slice(2, 3);
< ▶ ['blue']
> colors.slice(-2);
< ▶ (2) ['pink', 'white']
```





APNA COLLEGE

# Array Methods

```
> let colors = ["red", "yellow", "blue", "orange", "pink", "white"];
```

**splice : removes / replaces / add elements in place**

**splice( start, deleteCount, item0...itemN)**

```
> colors.splice(4);
< ▶ (2) ['pink', 'white']
> colors
< ▶ (4) ['red', 'yellow', 'blue', 'orange']
> colors.splice(0, 1);
< ▶ ['red']
> colors
< ▶ (3) ['yellow', 'blue', 'orange']
> colors.splice(0, 1, "black", "grey");
< ▶ ['yellow']
> colors
```

# Array Methods

sort : sorts an array

ascending
descending

```
> let days = ["monday", "sunday", "wednesday", "tuesday"];
< undefined
> days.sort();
< ▶ (4) ['monday', 'sunday', 'tuesday', 'wednesday']
```

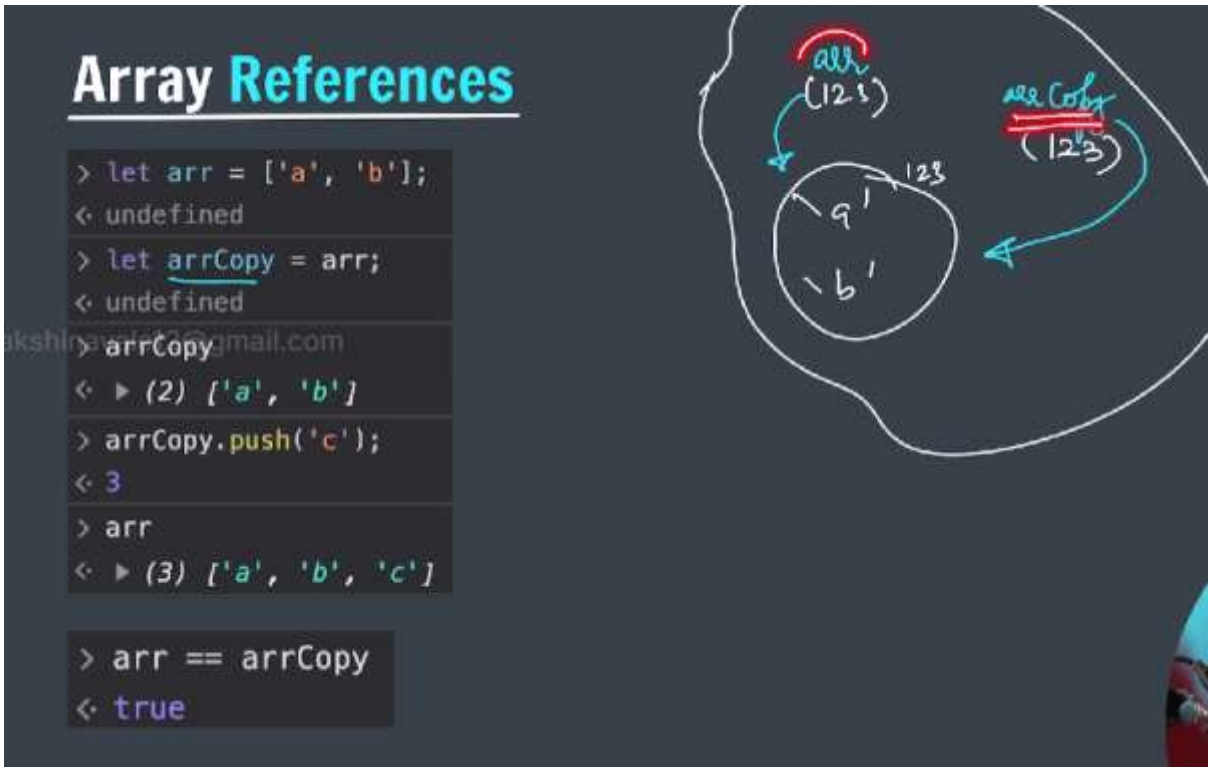
```
> let squares = [25, 16, 4, 49, 36, 9]
< undefined
> squares.sort();
< ▶ (6) [16, 25, 36, 4, 49, 9]
```

# Array References

address in memory

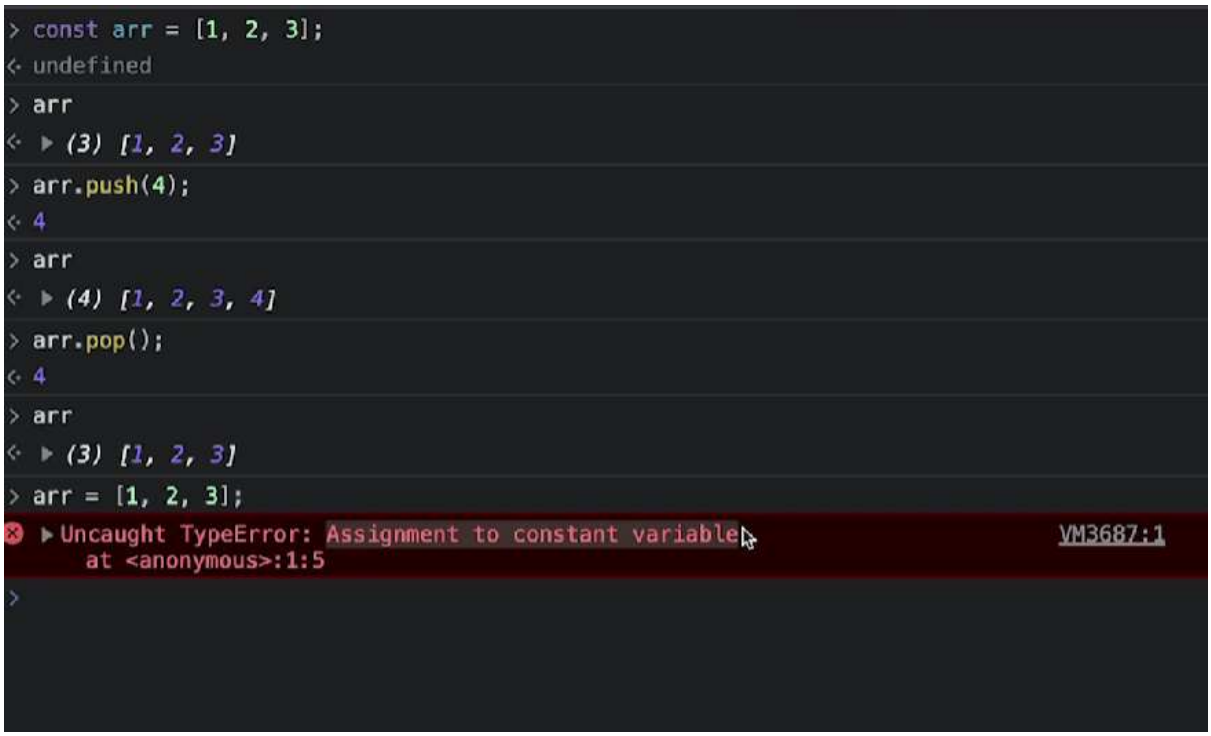
```
> [1] === [1]
< false
> [1] == [1]
< false
```

```
> "name" == "name"
< true
> "name" === "name"
< true
> [1] === [1]
< false
> [1] == [1]
< false
> [] == []
< false
> [] === []
< false
>
```



# Constant Arrays

```
const arr = [1, 2, 3, 4];
```



# Nested Arrays → multi dimensional

## array of arrays

```
> let nums = [ [2, 4], [3, 6], [4, 8] ];
< undefined
> nums
< ▼ (3) [Array(2), Array(2), Array(2)]
    ▶ 0: (2) [2, 4]
    ▶ 1: (2) [3, 6]
    ▶ 2: (2) [4, 8]
      length: 3
    ▶ [[Prototype]]: Array(0)
```

# Nested Arrays

nums[1][1]

```
> let nums = [ [2, 4], [3, 6], [4, 8] ];
```

| 0,0 | 0,1 |
|---|---|
| 2 | 4 |
| 3 | 6 |
| 4 | 8 |

1,0 1,1 2,0 2,1

nums[row][col]
nums[0][0]
nums[0][1]

nums[0][0] //2